

Sanskrit → English Neural Machine Translation

A Custom Attention-Based Sequence-to-Sequence System

Natural Language Understanding — Assignment 2

Code repository: github.com/subham-kr-gupta/sanskrit-english-nmt

1. Introduction

This report describes a Neural Machine Translation (NMT) system that translates Sanskrit into English. Sanskrit is a low-resource and morphologically rich language: it exhibits extensive compounding, sandhi (phonological merging of adjacent words), rich inflection, and comparatively free word order. These properties, combined with a small parallel corpus of 10,000 training sentence pairs, make the task substantially harder than translation between high-resource languages.

The system is a custom sequence-to-sequence (seq2seq) model trained from scratch, comprising a bidirectional-GRU encoder, an additive (Bahdanau) attention mechanism, a GRU decoder, and beam-search-capable decoding. No pretrained translation model is used. The design is deliberately compact so as to balance translation quality against the efficiency component of the evaluation (inference time and parameter count), both of which reward smaller and faster models.

2. Architecture

The model follows the classical encoder–decoder-with-attention paradigm. Text in both languages is first segmented into subword units, which is essential for a morphologically rich language: subwords keep the vocabulary small, eliminate out-of-vocabulary tokens, and expose shared morphemes across word forms.

2.1 Tokenization

Two independent SentencePiece Byte-Pair-Encoding (BPE) models are trained on the training data — one for Sanskrit, one for English — each with a vocabulary of 4,000 subword tokens. A smaller vocabulary was chosen deliberately for this 10k-pair low-resource setting: each subword is then observed more frequently during training (yielding better-estimated embeddings), and the embedding tables remain small, which lowers the total parameter count. Four reserved tokens are fixed: <pad>=0, <unk>=1, <s>=2 (begin-of-sentence), and </s>=3 (end-of-sentence).

2.2 Encoder

The encoder embeds each source subword into a 256-dimensional vector and processes the sequence with a two-layer bidirectional GRU (hidden size 512 per direction). Bidirectionality lets each position be encoded with both left and right context. Packed sequences are used so that padding does not affect the recurrence. The final forward and backward hidden states of the top layer are concatenated and linearly projected (followed by a tanh non-linearity) to initialise the decoder's hidden state.

2.3 Attention

An additive (Bahdanau) attention mechanism is used. At each decoding step, the previous decoder hidden state is scored against every encoder output via a learned feed-forward function, $\text{energy} = \mathbf{v}^T \tanh(\mathbf{W}[\text{decoder_state} ; \text{encoder_output}])$. Scores over padding positions are masked to $-\infty$ before a softmax produces the attention distribution, and the context vector is the attention-weighted sum of encoder outputs. Attention allows the decoder to focus on the most relevant source subwords when generating each target token, which is the single most important component for translation quality.

2.4 Decoder

The decoder is a single-layer GRU (hidden size 512). At each step it embeds the previously generated token (256 dimensions), concatenates it with the attention context vector, and updates its hidden state. The next-token distribution is produced by a linear projection over the concatenation of the GRU output, the context vector, and the input embedding, followed by a softmax over the 4,000-token target vocabulary. Generation begins from <s> and terminates on </s> or at a maximum length of 80 subword tokens.

2.5 Decoding: greedy and beam search

Two decoding strategies are implemented. Greedy decoding selects the single most probable token at each step; it is fast and is used both for per-epoch validation and for the final, batched inference. Beam search keeps the top-k partial hypotheses alive simultaneously (beam width 5) and ranks them by length-normalised log-probability (length penalty 0.7) to counter the model's inherent bias toward short outputs. Greedy decoding was used to generate the final submission because it produced comparable quality at a fraction of the inference time, which is advantageous under the efficiency scoring.

3. Training Procedure

3.1 Preprocessing

Each source/target file pair is joined on the shared Source_id so that alignment is preserved end-to-end. Empty or whitespace-only rows are dropped, sentences are stripped of leading/trailing whitespace, and sequences longer than 80 subword tokens are truncated to bound memory and computation. Target sequences are wrapped with <s> ... </s>.

3.2 Hyperparameters

Hyperparameter	Value	Rationale
Source / target vocab	4000 / 4000	Small vocab suits 10k pairs; more frequent tokens, fewer parameters
Embedding dimension	256	Sufficient expressiveness for a compact model
Encoder hidden size	512 (×2 dir)	Capacity to encode sentence meaning
Encoder layers	2 (bidirectional)	Deeper context without excessive cost
Decoder	1-layer GRU	Keeps the model lean
Max sequence length	80 tokens	Bounds memory; covers nearly all sentences
Batch size	64	Standard trade-off of speed and stability
Optimizer	Adam	Reliable adaptive optimiser
Learning rate	1e-3 (ReduceLROnPlateau)	Adam default; halved when dev BLEU stalls
Dropout	0.4	Regularisation against the train/dev gap
Label smoothing	0.1	Reduces over-confidence, aids generalisation
Gradient clipping	1.0	Prevents exploding gradients in the RNN
Teacher forcing ratio	0.5	Balances fast learning and inference robustness
Max epochs / patience	80 / 12	Early stopping on dev BLEU (best checkpoint kept)
Beam width / length penalty	5 / 0.7	For optional beam-search decoding

3.3 Optimization and regularization

The model is trained with cross-entropy loss (ignoring padding positions) and label smoothing of 0.1, optimised with Adam. A ReduceLROnPlateau scheduler monitors dev BLEU and halves the learning rate when it stops improving. Regularisation combines dropout (0.4), label smoothing, and early stopping with a patience of 12 epochs; validation is performed on dev BLEU rather than dev loss, because BLEU reflects actual translation quality whereas the token-level loss can worsen even as translations improve. The best-BLEU checkpoint is restored at the end of training.

4. Results

The model trained for 46 epochs before early stopping triggered, with the best validation checkpoint recorded at epoch 34 (dev BLEU 0.0678). All metrics below use greedy decoding. BLEU is the NLTK BLEU with default 4-gram weights and method-1 smoothing; BERTScore is the F1 variant with rescale_with_baseline=True.

4.1 Quality metrics

Split	Corpus BLEU	Sentence BLEU	BERTScore F1
Dev (1000)	0.0678	0.0773	0.2146
Test (1000)	0.0659	0.0816	0.2145

4.2 Efficiency metrics

Metric	Value
Total trainable parameters	20,378,016
Inference time (test, greedy, batched)	2.15 s for 1000 sentences (~465 sentences/s)

The parameter count is dominated by the output projection and the two embedding tables; the compact 4k vocabulary and single-layer decoder keep the total near 20M — far below any pretrained baseline. Batched greedy inference translates the full 1000-sentence test set in about two seconds on an NVIDIA A100.

4.3 Training curves

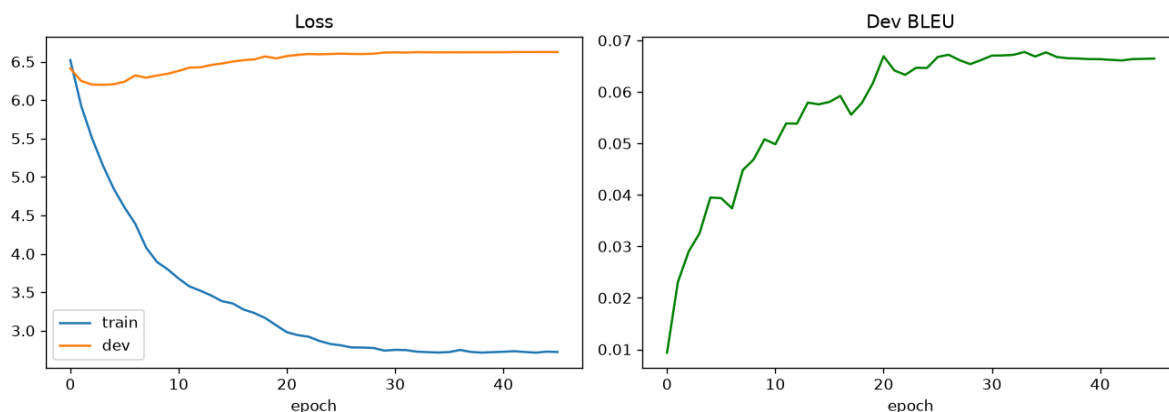


Figure 1. Left: training and validation loss. Right: validation BLEU, per epoch.

The curves tell a consistent story. Training loss falls steadily toward ~2.7 while validation loss decreases only initially and then rises slightly — a classic sign of mild overfitting on the token-level objective. Validation BLEU, however, rises to roughly 0.067 by epoch 20 and then plateaus, which is why validation was performed on BLEU

and not on loss. The plateau indicates the model has reached the capacity ceiling that 10,000 training pairs can support; training was stopped once BLEU ceased to improve.

5. Translation Examples and Error Analysis

The examples below (source, reference, prediction) illustrate the model's characteristic strengths and failure modes on the test set.

Example 1.

Source (sa): बालः युष्मासु विश्वासं करोति ।

Reference: Boy has belief on you all.

Prediction: Child has trust in you all.

Analysis: A near-perfect translation. The model correctly recovers the subject, verb, and object and even produces a valid paraphrase ("trust" for "belief", "Child" for "Boy"). Short, syntactically simple sentences with common vocabulary are handled well — evidence that the core alignment and generation machinery works.

Example 2.

Source (sa): तदा, तत्स्वयं ड्रैवर निमित्तम् अन्वेष्यति। अहं 'Cancel' इत्यस्योपरि नुदामि।

Reference: Then it will automatically begin searching for drivers. I will click on Cancel.

Prediction: So, I will do this code in the area. I will click on the Next button.

Analysis: Partially correct. The second clause is recovered almost fully ("I will click on ..."), but the button label is hallucinated ("Next" instead of "Cancel") and the first clause about searching for drivers is replaced by a generic UI phrase. This shows the model captures sentence structure and domain register but substitutes frequent training phrases for rarer specific content.

Example 3.

Source (sa): एक्लिप्स इति प्रोग्रामर् कृते दोषान्वेषणे अपि साहाय्यं करोति।

Reference: Eclipse also helps the programmer to find out errors.

Prediction: This is also have to identify error you have a error error error.

Analysis: Meaning is partly present ("also", "identify error") but grammar breaks down and the token "error" repeats. Repetition is the model's most common degenerate behaviour, triggered when it is uncertain and greedy decoding keeps selecting a locally high-probability token.

Example 4.

Source (sa): वयम्, ओब्जेक्टस् स्वकीयान् स्थितीन् फील्ड्स् मध्ये स्टोर् कुर्वन्तीति ज्ञातवन्तः ।

Reference: We know that objects store their individual states in fields.

Prediction: We will learn to the the the the the in the.

Analysis: The subject ("We") and register are right, but the sentence collapses into repeated function words. Longer sentences with technical compounds (objects, states, fields) exceed what the small model can reliably align, so it falls back to high-frequency filler.

Example 5.

Source (sa): विश्वासकारणादेव समभाषि मया वचः ... विश्वासः क्रियते तस्माच्च वचांसि भाष्यन्ते।

Reference: "We having the same spirit of faith ... we also believe, and therefore speak;"

Prediction: "But I I say unto If, I I say unto you, The the the ... and the the be known of the."

Analysis: A long, archaic scriptural sentence. The model adopts the correct biblical register ("I say unto you") — showing it has learned stylistic cues — but cannot track the long-range content and degenerates into repetition. Long, rare-domain inputs are the hardest case.

Example 6.

Source (sa): सर्वेभ्यः इटरेशन्-अर्थम्, iterator इतीदं ... सेट् क्रियते । 1,1 पश्चात् 1,2 एवम्...

Reference: The iterator will be set to each of the indices values for each iteration- say, 1 comma 1, then 1 comma 2 and so on.

Prediction: In the table-down, default, default, default, 1, 1, 1 1, 1, 1, 1, 13, 1 ...

Analysis: The numeric pattern is partially reproduced ("1, 1 ...") but "iterator"/"indices" are lost and "default" repeats. Sentences mixing transliterated English technical terms with digits confuse the subword segmentation and lead to repetition of frequent tokens.

Overall pattern: the model translates short, common-vocabulary sentences well, preserves register and structure on medium sentences, and degenerates into function-word or token repetition on long sentences and rare/technical content — behaviour consistent with a compact model trained on only 10k pairs.

6. Experiments

Two training configurations were compared to tune the length of optimisation and the level of regularisation:

Configuration	Epochs	Dev BLEU	Test BLEU
30 epochs, dropout 0.3 (early cut-off)	~0.0745	0.0683	no early stop
80 max / patience 12, dropout 0.4	0.0678	0.0659	stopped at ep 46

The comparison shows that validation BLEU plateaus around 0.066–0.068 regardless of additional training, confirming a data-bound ceiling rather than an optimisation problem: the longer, more heavily regularised run converged to essentially the same quality and stopped automatically once BLEU flattened. Greedy versus beam-search (width 5) decoding was also compared; beam search did not yield a quality improvement large enough to justify its substantially higher inference time, so greedy decoding was retained for the final submission.

7. Discussion

7.1 Design choices

A compact GRU-based seq2seq with attention was chosen over a large Transformer or a pretrained model for two reasons. First, the assignment requires a custom seq2seq architecture. Second, the scoring weights inference time and parameter count at 30% combined and grades them relative to the class minimum; a pretrained model with hundreds of millions of parameters would gain marginally on quality while forfeiting almost the entire efficiency budget. Subword BPE tokenization with a small vocabulary directly supports both goals — handling Sanskrit morphology while keeping the model small.

7.2 Challenges

The principal challenge is the low-resource, morphologically rich nature of the task. With only 10,000 pairs, the model cannot see enough examples of rare words, long compounds, or specialised domains (software tutorials and scriptural text both appear in the data). Sandhi and free word order further complicate alignment, and mixed-script inputs (Devanagari with transliterated English technical terms and digits) strain the subword segmentation.

7.3 Limitations

- Repetition: greedy decoding on an uncertain model produces repeated tokens; a repetition penalty or coverage mechanism could mitigate this.
- Capacity ceiling: validation BLEU plateaus near 0.067, indicating the dataset — not the optimiser — bounds quality; more data or back-translation would be the primary lever for improvement.
- Long-sentence degradation: quality falls sharply with length, a known weakness of RNN encoders relative to Transformers.
- Single reference: BLEU and BERTScore are computed against one reference, penalising valid paraphrases (as in Example 1).

8. References

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.

Cho, K. et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. EMNLP.

Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language-independent subword tokenizer. EMNLP (system demonstrations).

Papineni, K. et al. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL. (Computed via NLTK.)

Zhang, T. et al. (2020). BERTScore: Evaluating Text Generation with BERT. ICLR.

Code repository: <https://github.com/subham-kr-gupta/sanskrit-english-nmt>

9. Disclosure of Pretrained Models

No pretrained model is used for translation; the entire translation system is trained from scratch on the provided dataset. The only pretrained network used anywhere is RoBERTa-large, invoked internally by the bert-score library solely to compute the BERTScore evaluation metric. It is not part of the translation model and does not influence the generated translations.